Inductive Types in Depth and Lean 4

Lecture 18

Section 1

Part I: The Problem

Natural numbers as a signature

In λC the only primitive is the Π-type. Suppose we want to reason about natural numbers. The naive approach: introduce a **signature**:

$$\Gamma_S \equiv N : \mathsf{Type},\ z : N,\ S : N \to N$$

We can derive: $\Gamma_S \vdash z : N,\ \Gamma_S \vdash S\ z : N,\ \Gamma_S \vdash S\ (S\ z) : N.$

We can also add an ordering relation with axioms:

$$\begin{aligned}\Gamma_N \equiv \Gamma_S, \mathsf{le} &: N \to N \to \mathsf{Prop},\\ \mathsf{le}_z &: \forall x.\, \mathsf{le}\ z\ x,\\ \mathsf{le}_S &: \forall x\ y.\, \mathsf{le}\ x\ y \to \mathsf{le}\ (S\ x)\ (S\ y)\end{aligned}$$

The context is too weak

For every numeral $S^n(z)$ we can build a proof of le $(S^n z)$ $(S^n z)$.

But can we prove $\forall x{:}N.\,\mathsf{le}\ x\ x$? **No.** The context only says N is *some* type with an element z and a function S. It does not say every element of N has the form $S^n(z)$. There could be extra elements, and le might not hold for them.

We need to add an induction principle. In first-order logic, you would need a separate axiom for each property P. In λC, one axiom covers all properties at once, because we can quantify over P:

$$\begin{aligned}&\mathsf{ind} : \Pi P{:}(N \to \mathsf{Prop}).\\ &\qquad P\ z \to (\Pi x.\, P\ x \to P\ (S\ x)) \to \Pi x.\, P\ x\end{aligned}$$

This axiom *asserts* that every element of N is reachable from z by applying S. With it in the context, $\forall x.\,\mathsf{le}\ x\ x$ becomes provable.

Using the induction axiom

With ind in the context, we can prove $\forall x.\, \mathsf{le}\ x\ x$:

$$\mathsf{ind}\ (\lambda x.\, \mathsf{le}\ x\ x)\ (\mathsf{le}_z\ z)\ (\lambda x\ h.\, \mathsf{le}_S\ x\ x\ h)$$

This applies ind to:

- the predicate $\lambda x.\, \mathsf{le}\ x\ x$
- the base case $\mathsf{le}_z\ z : \mathsf{le}\ z\ z$
- the step $\lambda x.\, \lambda h.\, \mathsf{le}_S\ x\ x\ h$

What the axiom approach still cannot do

Adding ind gives us induction. But:

1. **No pattern matching.** We cannot define pred by cases.
2. **No computation.** ind $P\ p_0\ s\ (S\ (S\ z))$ does not reduce.
3. **No disjointness or injectivity.** Cannot prove $S\ x \neq z$.
4. **Risk of inconsistency.** Arbitrary axioms may have no model.

We need a single mechanism that gives us all of this at once. That mechanism is **inductive definitions**.

Section 2

Part II: Inductive Definitions

Nat as an inductive type

```
inductive N : Type where
  | z : N
  | S : N → N
```

This adds $N : \text{Type}$, $z : N$, $S : N \to N$, the same constants as the signature. But the declaration asserts something more: **every element of N is built by z and S, and nothing else**.

What "nothing else'' gives us

Because every element is either z or $S\ k$:

- We can define functions by **pattern matching**: handle z and $S\ k$ separately, and all cases are covered.
- We can prove **induction**: any property that holds for z and is preserved by S holds for all of N.
- We get **disjointness** ($S\ x \neq z$) and **injectivity** ($S\ x = S\ y \to x = y$): the constructors are distinct and non-overlapping.

The system generates a recursor, computation rules, and these facts automatically. None of this was available from the signature.

Inductive relations: le

Inductive definitions are not just for data types:

```
inductive le : N → N → Prop where
  | lez (x : N) : le N.z x
  | leS (x y : N) (h : le x y)
      : le (N.S x) (N.S y)
```

Now le is defined as the **smallest relation** satisfying these rules. What does "smallest'' mean? It means every proof of le x y was built by one of these two constructors. There is no other way. So when proving a proposition about x and y, we can match on H : le x y and ask: was H built by lez or leS?

Note: the sort of N is Type (it defines data). The sort of le is Prop (it defines a proposition, the statement that $x \leq y$).

Inductive definitions are not axioms

The constructors of le look identical to the axioms le_z and le_S from Part I. So what changed?

	Signature (Part I)	**Inductive def.**
Status	assumption	construction
Content	"assume le satisfies these"	"build the smallest such le"
Consistency	might have no model	guaranteed to exist
Computation	inert axiom	reduces (computes)

An axiom says: *assume something exists*. An inductive definition says: *here it is, I am constructing it*. Because these are constructions, we can go further and **parameterize** them.

Parameterized inductive definitions

Since inductive definitions are constructions, we can parameterize them: build a new relation from any inputs A and R. With axioms, you would need a separate axiom schema for each choice.

```
inductive RT (A : Type)
    (R : A → A → Prop) : A → A → Prop where
  | refl  (x : A) : RT A R x x
  | step  (x y : A) (h : R x y) : RT A R x y
  | trans (x y z : A)
      (h1 : RT A R x z) (h2 : RT A R z y)
      : RT A R x y
```

This defines the reflexive-transitive closure of R: the smallest relation on A that is reflexive, transitive, and includes every pair in R. The inductive declaration *constructs* this relation. It always exists.

Logical connectives as inductive types

```
-- Absurdity ( ): no constructor, so no proof exists
inductive False : Prop where

-- Existential quantification
inductive ex (A : Type) (P : A → Prop)
    : Prop where
  | intro (w : A) (h : P w) : ex A P

-- Equality (Lean's built-in Eq is defined this way)
inductive Eq {A : Type} (a : A) : A → Prop where
  | refl : Eq a a    -- written a = a
```

Compare with the Π-type encodings from Lecture 16 ($\bot \equiv \Pi C{:}\mathsf{Prop}.\, C$, etc.). The inductive versions support pattern matching and computation; the encodings did not.

Anatomy of a declaration: Nat

What each part of a declaration is called:

```
inductive N : Type where    -- name: N, sort: Type
  | z : N                   -- constructor, no arguments
  | S : N → N               -- one argument of type N
```

Nat has no parameters and no indices. The sort is Type. The constructor S has one **constructor argument** of type N (the recursive occurrence).

Anatomy of a declaration: le

```
inductive le : N → N → Prop where
  -- sort: Prop, two indices (both N)
  | lez (x : N) : le N.z x
      -- index 1 fixed to z
  | leS (x y : N) (h : le x y)
      : le (N.S x) (N.S y)
      -- recursive argument: h
```

No parameters. Two **indices** of type N. They vary between constructors (z and x in lez; $S\,x$ and $S\,y$ in leS). The sort is Prop: this defines a relation, not data.

Anatomy of a declaration: eq

```
inductive Eq {A : Type} (a : A) : A → Prop where
  -- A implicit, a parameter, second A is index
  | refl : Eq a a  -- index forced to equal a
```

The parameter a is the same in every constructor. The index can vary, but refl constrains it to equal a.

Rule of thumb in Lean 4: arguments *before* the colon are parameters; arguments *after* the colon are indices.

Parameters vs. indices

Declaration	**Parameters**	**Indices**
`inductive N : Type`	(none)	(none)
`inductive List (A) : Type`	A	(none)
`inductive Vec (A) : N → Type`	A	$n : N$
`inductive le : N → N → Prop`	(none)	x, y
`inductive Eq (a : A) : A → Prop`	a	$b : A$

Parameters are fixed across all constructors. Indices can differ, and when you match, the type checker **learns what the index must be**. That is how refl forces $b = a$.

The positivity condition

Not every constructor is safe. Consider each constructor argument type (the types before the final target I ...). The rule is simple: **the type I being defined must not appear in the domain of any function type** within a constructor argument.

Constructor	**Argument types**	
S : Nat $\to$ Nat	Nat (just I, not inside $\to$)	✓
vcons : $A \to$ Vec $A\ n \to$ Vec $A\ (S\ n)$	A and Vec $A\ n$ (just I)	✓
intro : (Bad $\to$ Bad) $\to$ Bad	Bad $\to$ Bad (I in domain)	✗

In the first two, the argument is just `give me an existing $\Nat$''` orgive me an existing Vec.'' In the third, the argument is "give me a *function on* Bad,'' which enables self-application.

Why the positivity condition matters

The point is that every constructor argument should ask for data that already exists, not for a function that can call back into the type being defined. If a constructor takes a function $I \to \ldots$, then a value of I can contain a function that operates on values of I, including itself. That is what produced the non-terminating term $\omega\,(\text{intro}(\omega))$ in Lecture 17.

This restriction is called **strict positivity**. It guarantees that every inductive definition has a model and that the recursor terminates.

Section 3

Part III: Pattern Matching

Using inductive types: pattern matching

So far we have declared inductive types and their constructors (ways to **build** values). Now we need a way to **use** them: given a value of an inductive type, examine which constructor produced it and act accordingly. This is **pattern matching**.

```
def pred (x : N) : N :=
  match x with
  | N.z   => N.z
  | N.S n => n
```

One branch per constructor, and the match is exhaustive (the "nothing else'' property guarantees two branches cover all cases). When Lean evaluates pred $(S\ (S\ z))$, the argument $S\ (S\ z)$ matches the pattern $S\ n$ with $n = S\ z$, so the whole expression reduces to the body of that branch, which is n, giving $S\ z$. This kind of reduction, where a match sees which constructor was used and substitutes its arguments into the corresponding branch, is called ι-**reduction**.

How matching interacts with indices

Recall that Eq has a **parameter** a and an **index** b:

```
inductive Eq {A : Type} (a : A) : A → Prop where
  | refl : Eq a a   -- index constrained to equal a
```

The only constructor refl has type Eq a a, so the index must equal the parameter. What happens when we match on a proof $h : a = b$?

Equality elimination: matching on refl

To match $h : a = b$ against refl, the type checker must unify Eq a b (the type of h) with Eq a a (the type of refl). This forces $b = a$.

Inside the refl branch, b has been replaced by a. The goal $P\ b$ becomes $P\ a$:

```
theorem subst {a b : A}
    (P : A → Prop) (h : a = b) (pa : P a)
    : P b :=
  match h with
  | Eq.refl => pa  -- b is now a, goal is P a
```

From $a = b$ and $P\ a$, we conclude $P\ b$: equality lets us substitute equals for equals. (This elimination principle is called the **J-rule** in Martin-L"of type theory.)

Equality elimination: consequences

The same mechanism gives us symmetry, transitivity, and congruence:

```
theorem symm (h : a = b) : b = a :=
  match h with
  | Eq.refl => Eq.refl  -- b becomes a, goal: a = a

theorem trans (h1 : a = b) (h2 : b = c) : a = c :=
  match h2 with
  | Eq.refl => h1  -- c becomes b, goal: a = b

theorem congr (f : A → B) (h : a = b) : f a = f b :=
  match h with
  | Eq.refl => Eq.refl  -- b becomes a, goal: f a = f a
```

In each case, matching on refl unifies the two sides, simplifying the goal.

Inversion: when no constructor fits

Consider le with constructors:

$$\mathsf{lez}\ (x) : \mathsf{le}\ z\ x \qquad \mathsf{leS}\ (x)\ (y)\ (h : \mathsf{le}\ x\ y) : \mathsf{le}\ (S\ x)\ (S\ y)$$

Can we prove $\mathsf{le}\ (S\ n)\ z \to \bot$? Try matching on a proof $H : \mathsf{le}\ (S\ n)\ z$. The type checker checks each constructor:

- lez: produces $\mathsf{le}\ \mathbf{z}\ x$. First index is z, but we need $S\ n$. Since $z \neq S\ n$: **impossible**.
- leS: produces $\mathsf{le}\ (S\ x)\ (\mathbf{S}\ y)$. Second index is $S\ y$, but we need z. Since $S\ y \neq z$: **impossible**.

Inversion: zero cases, any conclusion

Both constructors are ruled out by index mismatch. The match has **zero** branches:

```
theorem le_inv (n : N) (H : le (N.S n) N.z)
    : False :=   -- False is  : no constructors
  match H with
  -- zero branches: no constructor produces le (S n) z
```

Every element of le $(S\ n)\ z$ must come from some constructor, but no constructor can produce one. The type is empty, so we can conclude $\bot$.

This follows from the "nothing else'' property: if no constructor can build a term of a given type, that type is uninhabited.

Section 4

Part IV: Delivering on the Promises

Recursive functions that compute

In Part I we saw that ind $P\ p_0\ s\ (S\ (S\ z))$ was inert. With inductive types, we can define recursive functions that actually reduce:

```
def add (m n : N) : N :=
  match n with
  | N.z   => m
  | N.S k => N.S (add m k)
-- Lean checks: k is a subterm of N.S k (termination)
```

When Lean sees add $(S\ z)\ (S\ (S\ z))$, it matches the second argument against $S\ k$ with $k = S\ z$, so the result is S (add $(S\ z)\ (S\ z)$). Then it matches $S\ z$ against $S\ k$ with $k = z$, giving $S\ (S$ (add $(S\ z)\ z)$). Finally it matches z against z, giving $S\ (S\ (S\ z))$. Each step fires because the matched argument starts with a constructor.

The proof we couldn't build before

In Part I, we wanted to prove $\forall x.\, \mathsf{le}\ x\ x$ but the context was too weak. Now we can:

```
theorem le_refl : (n : N) → le n n :=
  fun n => match n with
  | N.z   => le.lez N.z
  | N.S k => le.leS k k (le_refl k)
```

The recursive call gives $\mathsf{le}\ k\ k$ (the induction hypothesis); leS lifts it to $\mathsf{le}\ (S\ k)\ (S\ k)$. Recursion over data and induction over proofs are the same mechanism: pattern matching + a recursive call on a smaller argument.

Section 5

Conclusion

Properties of CIC

- **Strong normalization.** Every well-typed term has a normal form.
- **Decidable type-checking.** Follows from normalization.
- **Canonicity.** Every closed term of inductive type reduces to a constructor.
- **Consistency.** False has no constructors, so no closed proof of $\bot$ exists.

The positivity condition and the structural recursion check are both essential: they prevent the non-termination that would make every type inhabited.

What CIC gives us

Feature	**Signatures**	**Inductive defs**
Constructors	context vars	primitives
Pattern matching	impossible	built-in
Computation	none	ι-reduction
Recursion	impossible	fixpoints
Induction	axiom (inert)	derived (computes)
$S\ x \neq z$	unprovable	automatic
Consistency	at risk	positivity $\rightarrow$ model

What we covered

1. **The problem**: signatures give no matching, no computation, no induction, and risk inconsistency.
2. **The solution**: inductive definitions: every element comes from a constructor, and nothing else.
3. **Anatomy**: parameters vs. indices, the positivity condition (three cases, checked on the whole argument type).
4. **Pattern matching**: exhaustive case splits, equality elimination via index unification, inversion when no constructor fits.
5. **Recursion**: fixpoints with structural recursion. The payoff: unlike axioms, inductive definitions compute.

Next time

Next lecture: CIC in practice.

Dependent types in programming (vectors, safe indexing, the gap between definitional and propositional equality), the Prop elimination restriction, well-founded recursion, and type classes.